

Experiment No. 7

Student Name: Ojaswini Thote

PRN: 24070521048

Experiment Title

Implementation of DOM Manipulation and Event Handling using JavaScript; Development of a To-Do List and Profile Creation Form

Software / Tools Required

1. Visual Studio Code
2. Google Chrome
3. HTML5
4. JavaScript (ES6)

Practical — To-Do List / Task List

Aim

To develop an interactive To-Do List using JavaScript DOM manipulation and event handling, with functionality to add, edit, and delete tasks.

File Path

<file:///Users/ojaswinithote/Desktop/Javascript/EXP%207/Case%20Study/index1.html>

<file:///Users/ojaswinithote/Desktop/Javascript/EXP%207/Practical/index.html>

Program Code

Practical/index.html

```
<!DOCTYPE html>

<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Task List | To-Do List</title>

  ...
</style>
  @import
url('https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600;700&family=Playfair+Display:wght@600;700&display=swap');

  * {
    box-sizing: border-box;
  }

  body {
    margin: 0;
    min-height: 100vh;
    font-family: "DM Sans", sans-serif;
    display: flex;
    justify-content: center;
```

```

        align-items: center;
        padding: 30px 20px;
        background:
            radial-gradient(circle at 10% 10%, #e7ddff 0, transparent 30%),
            radial-gradient(circle at 90% 90%, #dceeff 0, transparent 30%),
            #f8f6f1;
    }

    .container {
        width: 100%;
        max-width: 600px;
        background: rgba(255, 255, 255, 0.82);
        border: 1px solid #e7e1d8;
        border-radius: 24px;
        padding: 38px;
        box-shadow: 0 18px 45px rgba(109, 94, 133, 0.12);
    }

    .header {
        display: flex;
        align-items: center;
        justify-content: space-between;
        margin-bottom: 30px;
        padding-bottom: 20px;
        border-bottom: 1px solid #ebe6df;
    }

    h1 {
        font-family: "Playfair Display", serif;
        font-size: 32px;
        color: #51466b;
        margin: 0 0 5px;
    }

    .header p {
        color: #8a8491;
        margin: 0;
        font-size: 14px;
    }

    .date-box {
        background: #eee9f8;
        color: #695b84;
        padding: 10px 14px;
        border-radius: 12px;
        font-size: 13px;
        font-weight: 600;
    }

    .input-section {
        display: flex;
        gap: 12px;
        margin-bottom: 25px;
    }

    input {
        flex: 1;
        padding: 14px 16px;
        border: 1.5px solid #ded6ea;
        background: #fcfbfd;
        border-radius: 12px;
        font-family: inherit;
        font-size: 15px;
        color: #51466b;
        outline: none;
        transition: 0.25s;
    }

    input::placeholder {

```

```

        color: #aaa4b0;
    }

    input:focus {
        border-color: #a895c7;
        box-shadow: 0 0 0 4px rgba(168, 149, 199, 0.12);
    }

    #addBtn {
        border: none;
        background: #a895c7;
        color: white;
        padding: 0 22px;
        border-radius: 12px;
        font-family: inherit;
        font-size: 14px;
        font-weight: 600;
        cursor: pointer;
        transition: 0.25s;
    }

    #addBtn:hover {
        background: #8f7ab2;
        transform: translateY(-2px);
        box-shadow: 0 6px 14px rgba(143, 122, 178, 0.2);
    }

    .task-info {
        display: flex;
        justify-content: space-between;
        align-items: center;
        margin-bottom: 14px;
    }

    .task-info h2 {
        font-family: "Playfair Display", serif;
        font-size: 20px;
        color: #5c5368;
        margin: 0;
    }

    .task-count {
        background: #e8f1fa;
        color: #6c8196;
        padding: 6px 11px;
        border-radius: 20px;
        font-size: 12px;
        font-weight: 600;
    }

    ul {
        list-style: none;
        padding: 0;
        margin: 0;
    }

    li {
        display: flex;
        align-items: center;
        gap: 12px;
        padding: 15px;
        margin-bottom: 11px;
        background: #faf8fc;
        border: 1px solid #eee9f2;
        border-radius: 14px;
        animation: appear 0.35s ease;
        transition: 0.25s;
    }

```

```

li:hover {
  background: #f6f2fa;
  transform: translateY(-2px);
  box-shadow: 0 7px 15px rgba(99, 83, 120, 0.07);
}

.task-number {
  width: 28px;
  height: 28px;
  min-width: 28px;
  display: flex;
  justify-content: center;
  align-items: center;
  border-radius: 50%;
  background: #e9e1f4;
  color: #7c6a96;
  font-size: 12px;
  font-weight: 700;
}

.task-text {
  flex: 1;
  color: #554d60;
  font-size: 15px;
  word-break: break-word;
}

.edit-btn,
.delete-btn {
  border: none;
  padding: 8px 12px;
  border-radius: 9px;
  font-family: inherit;
  font-size: 12px;
  font-weight: 600;
  cursor: pointer;
  transition: 0.2s;
}

.edit-btn {
  background: #e7f0f7;
  color: #5d788f;
}

.edit-btn:hover {
  background: #d8e8f2;
}

.delete-btn {
  background: #f8e8e8;
  color: #a56c72;
}

.delete-btn:hover {
  background: #f2d9da;
}

.empty-message {
  text-align: center;
  color: #aaa4b0;
  padding: 30px 10px;
  border: 1px dashed #ddd5e4;
  border-radius: 14px;
  font-size: 14px;
}

@keyframes appear {
  from {
    opacity: 0;
  }
}

```

```

        transform: translateY(10px);
    }

    to {
        opacity: 1;
        transform: translateY(0);
    }
}

@media (max-width: 550px) {
    .container {
        padding: 25px 20px;
    }

    .header {
        align-items: flex-start;
        flex-direction: column;
        gap: 15px;
    }

    .input-section {
        flex-direction: column;
    }

    #addBtn {
        padding: 14px;
    }

    .edit-btn,
    .delete-btn {
        padding: 7px 9px;
    }
}
</style>
...

</head>

<body>
...
<div class="container">

    <div class="header">
        <div>
            <h1>My Tasks</h1>
            <p>Organize your day, one task at a time.</p>
        </div>

        <div class="date-box" id="currentDate"></div>
    </div>

    <div class="input-section">
        <input
            type="text"
            id="taskInput"
            placeholder="What needs to be done?"
        >
        <button id="addBtn">Add Task</button>
    </div>

    <div class="task-info">
        <h2>To-Do List</h2>
        <span class="task-count" id="taskCount">0 tasks</span>
    </div>

    <ul id="taskList"></ul>

    <div id="emptyMessage" class="empty-message">

```

```

        Your list is empty. Add a task to get started.
    </div>

</div>

<script src="script.js"></script>
...

</body>
</html>

```

Practical/script.js

```

const taskInput = document.getElementById("taskInput");
const addBtn = document.getElementById("addBtn");
const taskList = document.getElementById("taskList");
const emptyMessage = document.getElementById("emptyMessage");
const taskCount = document.getElementById("taskCount");
const currentDate = document.getElementById("currentDate");

const today = new Date();

currentDate.textContent = today.toLocaleDateString(
    "en-US",
    {
        month: "short",
        day: "numeric",
        year: "numeric"
    }
);

addBtn.addEventListener("click", addTask);

taskInput.addEventListener("keydown", function (event) {
    if (event.key === "Enter") {
        addTask();
    }
});

function addTask() {
    const task = taskInput.value.trim();

    if (task === "") {
        alert("Please enter a task.");
        taskInput.focus();
        return;
    }

    const li = document.createElement("li");

    const taskNumber = document.createElement("span");
    taskNumber.className = "task-number";

    const taskText = document.createElement("span");
    taskText.className = "task-text";
    taskText.textContent = task;

    const editBtn = document.createElement("button");
    editBtn.textContent = "Edit";
    editBtn.className = "edit-btn";

    const deleteBtn = document.createElement("button");
    deleteBtn.textContent = "Delete";
    deleteBtn.className = "delete-btn";

    editBtn.addEventListener("click", function () {
        const newTask = prompt(
            "Edit your task:",
            taskText.textContent

```

```

    });

    if (newTask !== null && newTask.trim() !== "") {
        taskText.textContent = newTask.trim();
    }
});

deleteBtn.addEventListener("click", function () {
    li.remove();
    updateTaskList();
});

li.appendChild(taskNumber);
li.appendChild(taskText);
li.appendChild(editBtn);
li.appendChild(deleteBtn);

taskList.appendChild(li);

taskInput.value = "";
taskInput.focus();

updateTaskList();
}

function updateTaskList() {
    const tasks = taskList.querySelectorAll("li");

    tasks.forEach(function (task, index) {
        const numberElement =
            task.querySelector(".task-number");

        numberElement.textContent = index + 1;
    });

    const count = tasks.length;

    taskCount.textContent =
        count + (count === 1 ? " task" : " tasks");

    if (count === 0) {
        emptyMessage.style.display = "block";
    } else {
        emptyMessage.style.display = "none";
    }
}

```

JavaScript Concepts / Functions Used

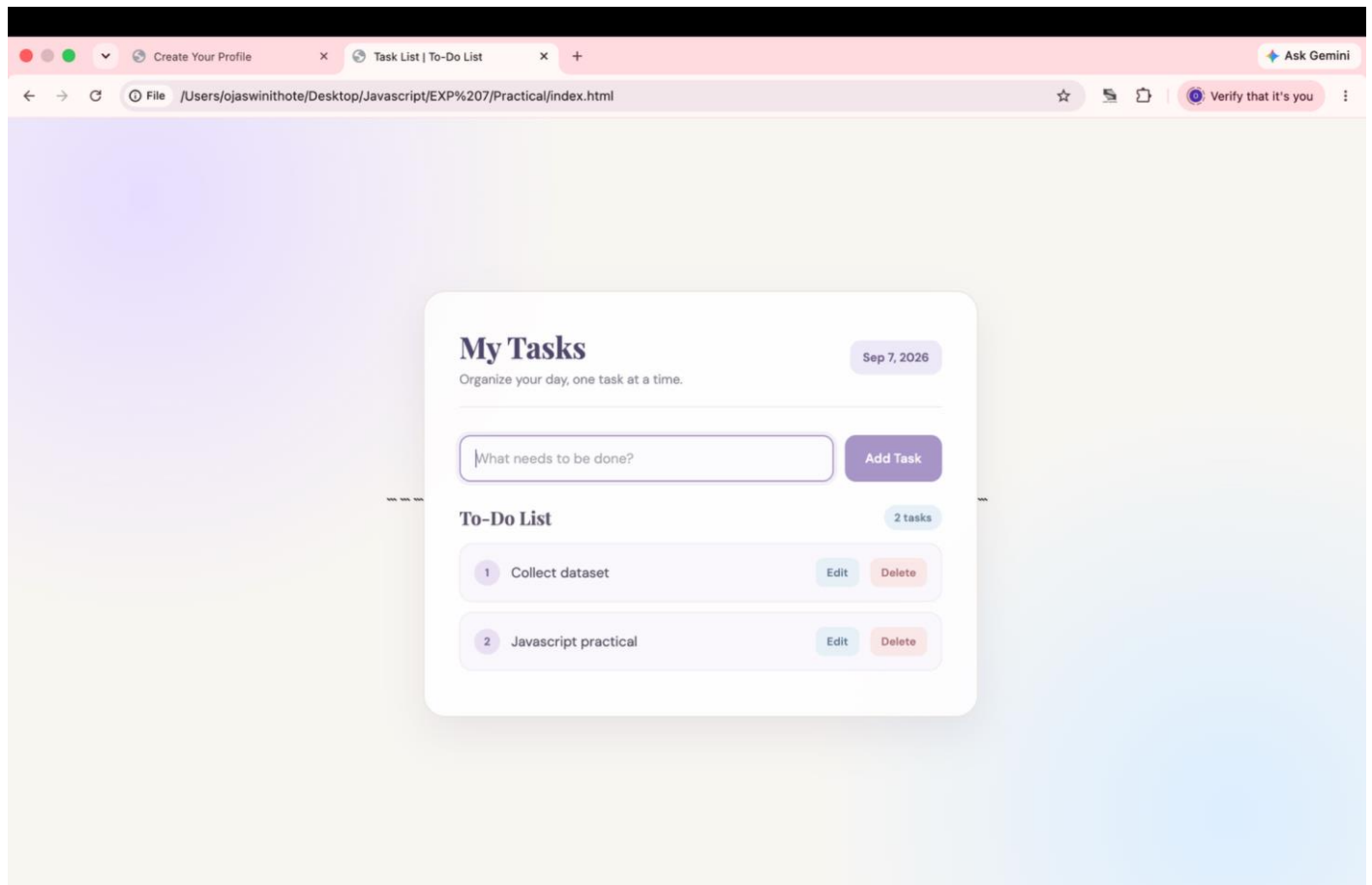
Function / Concept	Purpose
getElementById()	Accesses HTML elements from the DOM.
addEventListener()	Handles click and keyboard events.
createElement()	Creates task and button elements dynamically.
appendChild()	Adds dynamically created elements to the task list.
textContent	Sets and updates task text and task numbers.
remove()	Deletes a task from the list.
querySelectorAll()	Selects all task list items for counting and numbering.
Date / toLocaleDateString()	Displays the current date.
prompt()	Accepts edited task text from the user.

Output

- The current date is displayed in the task-list header.
- The user can enter a task and click Add Task to add it dynamically.

- Pressing Enter in the task input also adds the task.
- Each task receives a number and Edit and Delete buttons.
- The Edit button allows the task text to be modified.
- The Delete button removes the task and updates numbering and task count.
- An empty-list message is displayed when no tasks are present.

Screenshot



Case Study — Create Your Profile

Aim

To create a profile registration form using JavaScript event handling and form validation for required fields, email, password, website URL, date of birth, and terms and conditions.

File Path

Case Study/index1.html

Case Study/script1.js

Program Code

Case Study/index1.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Create Your Profile</title>

  <style>
    @import
url('https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600;700&family=Playfair+Display:wght@600;700&displa
y=swap');

    * {
      box-sizing: border-box;
    }

    body {
      margin: 0;
      min-height: 100vh;
      padding: 50px 20px;
      font-family: "DM Sans", sans-serif;

      background:
        radial-gradient(circle at 10% 20%, #d8cdf0 0, transparent 25%),
        radial-gradient(circle at 90% 80%, #c9dfef 0, transparent 28%),
        radial-gradient(circle at 85% 15%, #f1d8df 0, transparent 22%),
        #f3f1f7;

      display: flex;
      justify-content: center;
      align-items: center;
    }

    .container {
      width: 100%;
      max-width: 680px;
      background: #ffffff;
      border-radius: 24px;
      overflow: hidden;

      border: 1px solid #ded5e8;

      box-shadow:
        0 20px 50px rgba(84, 65, 112, 0.18),
        0 4px 12px rgba(84, 65, 112, 0.08);
    }

    .header {
      padding: 35px 42px 28px;

      background:
        linear-gradient(
          135deg,
          #e7ddf5,
          #dcebf5
        );

      border-bottom: 1px solid #d5cae3;
      position: relative;
    }

    .header::after {
      content: "\n";
    }
```

```

position: absolute;
right: 40px;
top: 25px;
font-size: 36px;
color: #8d72b5;
opacity: 0.7;
}

h1 {
  margin: 0 0 8px;
  font-family: "Playfair Display", serif;
  font-size: 34px;
  color: #4d3f66;
  letter-spacing: -0.5px;
}

.header p {
  margin: 0;
  color: #70667d;
  font-size: 15px;
}

.required-note {
  margin: 0;
  padding: 15px 42px;

  background: #faf7fc;

  border-bottom: 1px solid #eee7f2;

  color: #96707d;
  font-size: 13px;
  font-weight: 600;
}

form {
  padding: 30px 42px 42px;
}

.form-row {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 18px;
}

.form-group {
  margin-bottom: 20px;
}

label {
  display: block;
  margin-bottom: 8px;
  color: #514860;
  font-size: 14px;
  font-weight: 700;
}

.star {
  color: #bd6678;
}

.optional {
  color: #968f9e;
  font-size: 12px;
  font-weight: 500;
}

input,
select {

```

```

width: 100%;
padding: 13px 14px;

border: 1.5px solid #d9d0e2;
border-radius: 11px;

background: #fcfbfd;
color: #4e4659;

font-family: inherit;
font-size: 14px;

outline: none;

transition:
  border-color 0.2s,
  box-shadow 0.2s,
  transform 0.2s;
}

input:hover,
select:hover {
  border-color: #b8a6cf;
}

input:focus,
select:focus {
  background: #ffffff;
  border-color: #937ab6;

  box-shadow:
    0 0 0 4px rgba(147, 122, 182, 0.14);
}

.dob-row {
  display: grid;
  grid-template-columns: 1fr 1.4fr 1fr;
  gap: 10px;
}

.checkbox-group {
  display: flex;
  align-items: center;
  gap: 12px;

  margin: 25px 0;

  padding: 17px;

  background:
    linear-gradient(
      135deg,
      #f6f1fa,
      #f2f7fb
    );

  border: 1px solid #ded5e8;
  border-radius: 14px;
}

.checkbox-group input {
  width: 18px;
  height: 18px;

  margin: 0;

  accent-color: #8e73b3;
  cursor: pointer;
}

```

```

.checkbox-group label {
  margin: 0;
  cursor: pointer;
  font-weight: 600;
  font-size: 13px;
}

#submitBtn {
  width: 100%;

  padding: 15px;

  border: none;
  border-radius: 12px;

  background:
    linear-gradient(
      135deg,
      #8d72b5,
      #a482bd
    );

  color: white;

  font-family: inherit;
  font-size: 15px;
  font-weight: 700;

  letter-spacing: 0.2px;

  cursor: pointer;

  box-shadow:
    0 8px 18px rgba(141, 114, 181, 0.28);

  transition:
    transform 0.2s,
    box-shadow 0.2s;
}

#submitBtn:hover {
  transform: translateY(-2px);

  box-shadow:
    0 12px 24px rgba(141, 114, 181, 0.35);
}

#submitBtn:active {
  transform: translateY(0);
}

#message {
  display: none;

  margin: 20px 0 0;
  padding: 13px;

  border-radius: 10px;

  text-align: center;
  font-size: 14px;
  font-weight: 600;
}

.error {
  display: block !important;
  background: #f9e7ea;
  color: #a65365;
}

```

```

        border: 1px solid #edc8d0;
    }

    .success {
        display: block !important;
        background: #e6f2ea;
        color: #4f8060;
        border: 1px solid #c6dfce;
    }

    @media (max-width: 600px) {

        body {
            padding: 25px 15px;
        }

        .header {
            padding: 28px 25px;
        }

        form {
            padding: 25px;
        }

        .required-note {
            padding: 15px 25px;
        }

        .form-row {
            grid-template-columns: 1fr;
            gap: 0;
        }

        h1 {
            font-size: 28px;
        }
    }
</style>
</head>

<body>

    <div class="container">

        <div class="header">
            <h1>Create Your Profile</h1>
            <p>Enter your details to create your account.</p>
        </div>

        <p class="required-note">
            <span class="star">*</span> Required fields
        </p>

        <form id="registrationForm">

            <div class="form-row">

                <div class="form-group">
                    <label for="firstName">
                        First Name <span class="star">*</span>
                    </label>

                    <input
                        type="text"
                        id="firstName"
                        placeholder="Enter your first name"
                    >
                </div>
            </div>
        </form>
    </div>

```

```

        <div class="form-group">
            <label for="lastName">
                Last Name <span class="star">*</span>
            </label>

            <input
                type="text"
                id="lastName"
                placeholder="Enter your last name"
            >
        </div>

    </div>

    <div class="form-group">

        <label>
            Date of Birth <span class="star">*</span>
        </label>

        <div class="dob-row">

            <select id="day">
                <option value="">Day</option>
            </select>

            <select id="month">
                <option value="">Month</option>
            </select>

            <select id="year">
                <option value="">Year</option>
            </select>

        </div>

    </div>

    <div class="form-group">

        <label for="username">
            Username <span class="star">*</span>
        </label>

        <input
            type="text"
            id="username"
            placeholder="Choose a username"
        >

    </div>

    <div class="form-group">

        <label for="email">
            Email Address <span class="star">*</span>
        </label>

        <input
            type="email"
            id="email"
            placeholder="you@example.com"
        >

    </div>

```

```

<div class="form-group">

    <label for="password">
        Password <span class="star">*</span>
    </label>

    <input
        type="password"
        id="password"
        placeholder="Minimum 6 characters"
    >

</div>
<div class="form-group">

<label for="confirmPassword">
    Re-enter Password <span class="star">*</span>
</label>

<input
    type="password"
    id="confirmPassword"
    placeholder="Re-enter your password"
>

</div>

<div class="form-group">

    <label for="website">
        Personal Website
        <span class="optional">(Optional)</span>
    </label>

    <input
        type="url"
        id="website"
        placeholder="https://example.com"
    >

</div>

<div class="checkbox-group">

    <input type="checkbox" id="terms">

    <label for="terms">
        I agree to the
        <strong>Terms and Conditions</strong>
        <span class="star">*</span>
    </label>

</div>

<button type="submit" id="submitBtn">
    Create Profile
</button>

<div id="message"></div>

</form>

</div>

```

```
<script src="script1.js"></script>

</body>
</html>
```

Case Study/script1.js

```
const form = document.getElementById("registrationForm");

const firstName = document.getElementById("firstName");
const lastName = document.getElementById("lastName");
const day = document.getElementById("day");
const month = document.getElementById("month");
const year = document.getElementById("year");
const username = document.getElementById("username");
const email = document.getElementById("email");
const password = document.getElementById("password");
const confirmPassword = document.getElementById("confirmPassword");
const website = document.getElementById("website");
const terms = document.getElementById("terms");
const message = document.getElementById("message");

for (let i = 1; i <= 31; i++) {
    const option = document.createElement("option");
    option.value = i;
    option.textContent = i;
    day.appendChild(option);
}

const months = [
    "January",
    "February",
    "March",
    "April",
    "May",
    "June",
    "July",
    "August",
    "September",
    "October",
    "November",
    "December"
];

months.forEach(function (monthName, index) {
    const option = document.createElement("option");
    option.value = index + 1;
    option.textContent = monthName;
    month.appendChild(option);
});

const currentYear = new Date().getFullYear();

for (let i = currentYear; i >= 1950; i--) {
    const option = document.createElement("option");
    option.value = i;
    option.textContent = i;
    year.appendChild(option);
}

firstName.addEventListener("focus", function () {
    firstName.style.backgroundColor = "#f5f1f9";
});
```



```

firstName.addEventListener("blur", function () {
    firstName.style.backgroundColor = "#fcfbfd";
});

website.addEventListener("change", function () {

    if (website.value.trim() !== "") {

        website.style.backgroundColor = "#eee7f7";
        website.style.borderColor = "#8d72b5";

        showMessage(
            "Website field has been changed.",
            "success"
        );

    }

});

form.addEventListener("submit", function (event) {

    event.preventDefault();

    if (
        firstName.value.trim() === "" ||
        lastName.value.trim() === "" ||
        day.value === "" ||
        month.value === "" ||
        year.value === "" ||
        username.value.trim() === "" ||
        email.value.trim() === "" ||
        password.value.trim() === "" ||
        confirmPassword.value.trim() === ""
    ) {

        showMessage(
            "Please complete all required fields.",
            "error"
        );

        return;
    }

    const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

    if (!emailPattern.test(email.value)) {

        showMessage(
            "Please enter a valid email address.",
            "error"
        );

        email.focus();

        return;
    }

    if (password.value.length < 6) {

        showMessage(
            "Password must contain at least 6 characters.",

```

```

        "error"
    );

    password.focus();

    return;
}

if (password.value !== confirmPassword.value) {

    showMessage(
        "Passwords do not match. Please try again.",
        "error"
    );

    confirmPassword.focus();

    return;
}

if (website.value.trim() !== "") {

    try {

        new URL(website.value);

    } catch (error) {

        showMessage(
            "Please enter a valid website URL.",
            "error"
        );

        website.focus();

        return;
    }

}

if (!terms.checked) {

    showMessage(
        "Please agree to the Terms and Conditions.",
        "error"
    );

    return;
}

showMessage(
    "Profile created successfully!",
    "success"
);

});

function showMessage(text, type) {

    message.textContent = text;
    message.className = type;

}

```

JavaScript Concepts / Validation Used

Function / Concept	Purpose
createElement()	Creates day, month, and year options dynamically.
for loop	Generates day and year options.
forEach()	Adds month options from the months array.
addEventListener()	Handles focus, blur, change, and submit events.
preventDefault()	Prevents normal form submission.
Regular Expression	Validates the email address format.
length	Checks that the password has at least 6 characters.
new URL()	Validates the optional personal website URL.
showMessage()	Displays validation errors and success messages.

Output

- The form accepts first name, last name, date of birth, username, email, password, confirmation password, and optional website.
- Date, month, and year selections are generated dynamically.
- Required fields are checked before profile creation.
- The email address is validated using a regular expression.
- The password must contain at least 6 characters.
- The password and confirmation password must match.
- The optional website is validated using the URL constructor.
- The user must agree to the Terms and Conditions.
- A success message is displayed when all validations are satisfied.

Screenshot

The screenshot shows a web browser window with a single tab titled 'Create Your Profile'. The address bar displays the file path: `File /Users/ojaswinithote/Desktop/Javascript/EXP%207/Case%20Study/index1.html`. The browser interface includes standard navigation buttons (back, forward, refresh) and a search bar. The page content features a 'Create Your Profile' form with the following elements:

- Title:** 'Create Your Profile' with a subtitle 'Enter your details to create your account.'
- Required Fields Legend:** '* Required fields'
- Form Fields:**
 - First Name ***: Text input with value 'Ojaswini'.
 - Last Name ***: Text input with value 'Thote'.
 - Date of Birth ***: Three dropdown menus showing '30', 'July', and '2005'.
 - Username ***: Text input with value 'ojaswini'.
 - Email Address ***: Text input with value 'ojaswini@gmail.com'.
 - Password ***: Password input field with masked characters '*****'.
 - Re-enter Password ***: Password input field with masked characters '*****'.
 - Personal Website (Optional)**: Text input with value 'https://example.com'.
- Agreement:** A checkbox labeled 'I agree to the Terms and Conditions *' which is checked.
- Submit Button:** A purple button labeled 'Create Profile'.

Result / Conclusion

Both the Practical and Case Study were completed successfully. The Practical demonstrated DOM manipulation and event handling through an interactive To-Do List that supports adding, editing, and deleting tasks. The Case Study demonstrated JavaScript form handling and validation using event listeners, regular expressions, password checks, URL validation, and dynamic generation of date options. Thus, the experiment successfully demonstrated the practical use of JavaScript for creating interactive and validated web applications.